Day 4

Before Binary search cleared HashMap and HashSet

// Did Two sum and Contains Duplicate again myself
// with HashMap and HashSet

Binary Search Algorithm

Binary search is a search algo in a limited space.
Binary search require sorting → IMP

Core / code of Binary search

low/l } high/h } Changable.

```
while (low <= high)
{
    mid = (low + high)/2
    if (key == a[mid])
    {
        return mid;
    }
    else if (key < a[mid])
    {
        high = mid - 1;
        low = low;
    }
    else
    {
        low = mid + 1;
        high = high;
    }
}
```

Eg: Key = 13

	0	1	2	3	4	5	6	7	8
a →	3	5	6	8	12	15	16	19	21

low ↑ mid ↑ high ↑

$\frac{0+8}{2} = 4$

∵ mid < 13
So low = mid + 1

	5	6	7	8
	15	16	19	21

low ↑ mid ↑ high ↑

$\frac{5+8}{2} = \frac{13}{2} = 6$

∵ mid > 13
∴ high = mid - 1

	5
	15

low ↑ high ↑
mid

$\frac{5+5}{2} = \frac{10}{2} = 5$

∵ mid > 13
∴ high = mid - 1

But low = 5 and high = 4
and low cannot be greater than high.
∴ The loop stops and the element is not found.

Day 4

coding — Iterative — Leetcode 704

```
public class Searching {
    public static int binarySearch(int[] a, int key) {
        int l = 0, h = a.length - 1, mid = 0;

        while (l <= h) {
            mid = l + h / 2;
            if (key == a[mid]) {
                return mid;
            } else if (key < a[mid]) {
                h = mid - 1;
            } else {
                l = mid + 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {3, 5, 6, 8, 12, 15, 16, 19, 21};
        int key = 13;
        System.out.println(binarySearch(a, key));
    }
}
```

Tip:

→ If the array is not sorted then go for linear search as time complexity is O(n)

→ But if the array is sorted then go for binary search as time complexity is O(log n)

→ If sorted then used binary search then TC = O(n log n + log n)

Day 9

0 + (n-0)

Rough mark (Dry run)

l m h

[1, 3, 5, 6] Target = 2

l = 0

n = 3 3 > 2 → h → m-1 = 0

l h

[1, 3, 5, 6] 2 > 1 → l = m+1

h l

[1, 3, 5, 6]

l m h

[1, 3, 5, 6] T = 2

0 1 2 3

l m h

[1, 3, 5, 6]

0 1 2 3

l, h

[1, 3, 5, 6]

0 1 2 3

Q. Leetcode #278 First Bad Version.
(Def did not do ora well with this)
(check later)

(Very confusing sometimes)

```
int low = 1;
int high = n;
while (low < high){
    int mid = low + (high - low)/2;
    if( isBadVersion(mid)){
        high = mid;
    }else{
        low = mid + 1;
    }
}
return low;
```

Day 4

Recursive Binary Search (Generic code)

```
class Solution {
    public int search(int[] nums, int target){
        return binarySearch(nums, target, 0, nums.length - 1);
    }

    public int binarySearch(int[] nums, int target, int low, int high){

        if(low > high){
            return -1;
        }

        int mid = (low + high)/2;

        if( target == nums[mid]) {
            return mid;
        }
        else if ( target < nums[mid]){
            return binarySearch(nums, target, low, mid-1);
        }
        else
        {
            return binarySearch(nums, target, mid+1, high);
        }
    }
}
```

Time complexity → $O(\log_2 n)$

For overflow use long long

or $mid = low + \frac{(high - low)}{2}$

Lower Bound

Day 4

Lower bound searches for numbers >= target.

Eg: [1, 2, 5, 3, 7, 8, 9, 9, 9, 11]
Target = 1.
Here index 0 will be the answer.

Target = 4.
Here index 4 will be the answer = 7

Code

```
class solution {
    public int lowerBound(int[] nums, int target){
        int low = 0;
        int high = nums.length;

        while(low < high){
            int mid = (low + high)/2;
            if(nums[mid] >= target){
                high = mid;
            } else {
                low = mid + 1;
            }
        }
        return low;
    }
}
```

Time complexity → O(log₂n)

Do not use nums.length - 1 → wrong.

Upper Bound

Day 4

~~Lower~~ Upper bound searches for smallest index value > target

Eg:

arr[] = [2, 3, 6, 7, 8, 8, 11, 11, 11, 12]

x = 6

upper bound will be index 3 which is 7

same dont use nums.length -1

x = 11

upper bound will be index 9 which is 9.

x = 12

which will be hypothetical return lower.

→ leetcode 34 done

using both upper and lower bound

medium → Difficulty.

upper bound just changes

nums[mid] > target